

SENYALERT USER ANALYSIS REPORT

Selected Scenario: Scenario B - Small Team (5-20 Users)

1. SCENARIO SELECTION

Scenario B is the best fit for SenyAlert because the current system is intended for a small local safety or response team. It supports named administrator and user accounts, individual permissions, shared incident handling, and up to four camera inputs.

The number of cameras alone does not determine the number of users. However, the four-camera limit, local SQLite storage, local desktop dashboard, and single-workstation design all point to a small operational deployment rather than an enterprise system.

Scenario C was not selected even though SenyAlert includes role-based access control (RBAC) and audit logs. Those features improve accountability for a small team, but the system is not fully built to support 100 or more users, simultaneous multi-workstation access, centralized identity, or enterprise-scale event distribution. Scaling SenyAlert up to an enterprise system is an avenue for future research and development that is beyond the current scope.

The selected scenario describes 5-20 named team members who may use the same secured installation across roles or work shifts. It should not be interpreted as proof that operators can use separate computers simultaneously.

2. TONE AND DETAIL LEVEL

The User Manual uses direct, non-technical language because small-team users may include guards, reception personnel, supervisors, or other staff members who are not tech literate. Instructions are organized around visible screens and actions such as Sign in, Acknowledge, Resolve, and Save operator update.

The manual provides enough detail for normal daily work without explaining Java, Python, WebSockets, SQLite queries, or the hand-landmark algorithm. Technical troubleshooting is assigned to administrators or developers.

The manual also clearly states the system's limitations. It does not claim that SenyAlert automatically contacts emergency services or offers personal subscriptions, personal notification muting, or an in-app bug-report form. This prevents small-team users from relying on functions that do not exist.

3. EFFECT OF SCALABILITY ON USER EXPERIENCE

For a small team, SenyAlert's local design keeps the workflow simple. Users sign in to one dashboard, see one pending incident queue, and use consistent statuses and evidence. Permission-based accounts allow the team to divide responsibilities without introducing the complexity of an enterprise identity system.

The current four-camera limit also keeps the dashboard and response process manageable. Each camera has a clear ID and location, allowing operators to understand where an alert originated.

The limitation is that the current architecture does not scale horizontally. It uses a local WebSocket connection, local databases, and a desktop interface. Adding many cameras, multiple dispatch stations, or many simultaneous users would require a durable broker, centralized data service, shared authentication, conflict handling, and additional performance testing.

For the selected user count, the main usability concern is coordination rather than raw account count. Acknowledge, Resolve, operator notes, permissions, and audit history help prevent team members from losing track of who handled an incident.

4. MOST VALUABLE EVENT-HANDLING FEATURE

The most valuable feature for this user group is asynchronous two-stage processing.

When the vision engine detects a valid Signal for Help, it immediately sends a distress event. The dashboard can create and display a pending incident with its snapshot before

the post-event video has finished recording. When recording is complete, a second media-ready event updates the same incident.

This design matters to a small response team because operators can begin assessing and responding to a possible emergency without waiting for the entire video file. Evidence preparation continues in the background while the human workflow begins.

The shared event token connects the initial alert and later video update. However, complete retry idempotency still requires a unique database constraint and conflict-safe insert behavior before the system can claim production-grade duplicate protection.

5. DOCUMENTATION CHOICES FOR SCENARIO B

The assignment's Scenario B examples mention channel subscriptions, notification muting, and bug reporting. SenyAlert does not currently implement those exact features, so the manual maps them honestly:

- Camera IDs and locations are explained as operational event channels.
- The manual explains the installation-wide audible policy instead of inventing a personal mute button.
- A structured external problem-report template is provided because there is no in-app bug-report form.
- RBAC and audit history are included because they are useful to a small team, even though they are commonly associated with enterprise systems.

These choices keep the manual aligned with the selected number of users while remaining accurate to the implemented system.